CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-[Section]
System Integration and Architecture

System Design Document (SDD)

Project Title: GuildHall

Prepared By: Leanda, John Luis Catarina

Version: 2.0

Date: 3/3/2026
Status: Review

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	2/18/2026	John Luis Leanda	Initial draft	Draft
2.0	3/3/2026	John Luis Leanda	All sections completed	Review
2.1	3/4/2026	John Luis Leanda	Updated database to Supabase	Review
0.4	[Date]	[Your Name]	Added UI/UX designs	Review
0.5	[Date]	[Your Name]	Incorporated feedback	Revised
0.6	[Date]	[Your Name]	Final review and corrections	Final
1	[Date]	[Your Name]	Baseline version for development	Approved

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY..... 4

1.0 INTRODUCTION..... 5

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION..... 5

3.0 NON-FUNCTIONAL REQUIREMENTS..... 9

4.0 SYSTEM ARCHITECTURE..... 11

5.0 API CONTRACT & COMMUNICATION..... 12

6.0 DATABASE DESIGN..... 17

7.0 UI/UX DESIGN..... 18

8.0 PLAN..... 21

EXECUTIVE SUMMARY

1.1 Project Overview

GuildHall is a gamified, community service platform designed to facilitate local volunteering and mutual aid through an "Adventurer's Guild" theme. Users can post localized "Quests" (tasks or requests for help) and earn Experience Points (XP) upon completion. The ecosystem consists of a Spring Boot REST API, a Postgre relational database, a React Vite web dashboard for community management, and a Kotlin Android application.

1.2 Objectives

1. Develop a functional community service platform with authentication, quest creation and catalog, and quest acceptance.
2. Implement a leveling and XP system to encourage user engagement and local volunteering. Rank titles (e.g. Bronze, Silver, Gold, Mithril) are assigned based on XP thresholds defined in the backend.
3. Ensure seamless data synchronization between the Android mobile client (for field use) and the React web client (for management).
4. Allow admins to create "Guilds" which are dedicated groups separating different users. Users can then join these groups.
5. Develop a centralized Spring Boot backend that handles complex business logic, such as quest state transitions and user progression.
6. Implement JWT-based authentication to protect user data and ensure quest integrity across all integrated systems.

1.3 Scope

Included Features:

- User or "Adventurer" profiles featuring levels, XP bars, and "Rank" and "Skill" tags.
- Secure login using JWT and BCrypt hashing. Support for native user registration and Google OAuth integration.
- Quest Board (CRUD): Creating, viewing, bidding on, and completing quests.
- Grouping users and quests into communities or "Guilds" based on specific city or neighborhood boundaries.
- Admin or "Guildmaster" panel for global guild management and creation

- Real-time status updates.
- Responsive web interface
- Native Android mobile application
- Postgre schema to handle complex relationships between users, guilds, and quest logs.
- Image uploading
- External API integration: fetch random inspirational quotes (Quotable)
- Real-time (Polling or WebSocket) coordination between the Poster and the Adventurer (user).
- Payment system for accomplished quests
- Email sending (SMTP)

Excluded Features:

- Logistics or delivery of physical items
- Video chat
- Public global chat
- Map API integration

1.0 INTRODUCTION

1.1 Purpose

This document serves as the formal design specification for GuildHall, a gamified community service platform. It outlines the architectural framework, technical requirements, and integration strategies necessary to connect a Spring Boot backend, a React web frontend, and a Kotlin Android mobile application. This document ensures that the "Adventurer Guild" logic features are consistently implemented across all system tiers.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: GuildHall

Domain: Community Service / Hyper-local Volunteering

Primary Users: Small organizations like schools, Barangay communities, or freelancers

Problem Statement: Smaller communities in the Philippines often lack a centralized, engaging way to request small favors or find volunteering opportunities. Existing social media groups like Facebook groups are cluttered and lack accountability or incentive systems.
Solution: A themed "Adventurer's Guild" platform that gamifies real-world help through a Quest/XP system, providing a structured environment for community cooperation with dedicated mobile and web interfaces.

2.2 Core User Journeys

Journey 1: New Adventurer Registration & Onboarding

1. User visits the Kotlin Mobile App or React Web App.
2. User selects "Continue with Google" (Google OAuth Integration).
3. System verifies the Google Token, creates a new record in the `Users` table, and issues a GuildHall JWT.
4. System sends a "Welcome to the Guild" SMTP Email to the user's Gmail address.
5. User views their profile, initialized at Level 1 with 0 XP.

Journey 2: Posting a Quest

1. Poster (User) fills out the "Quest Commission" form to enter their quest details.
2. Poster can upload a file to attach to the quest such as a pdf for instructions, or an image (optional).
3. Poster sets the reward type to either "volunteer" or "paid". If paid, enters the "Reward" amount and proceeds to the Stripe Sandbox checkout. If volunteer, skip payment.
4. For paid quests, system records the payment and sets status to OPEN. For volunteer quests, status is set to OPEN immediately

Journey 3: Completing a Quest

1. Helper (User) accepts an OPEN quest. Its status changes to PENDING.
2. Helper (User) accomplishes the task. They can chat with the Poster when necessary for more details.
3. The Poster clicks "Confirm Completion."
4. System Logic Triggers:

- Payment is finalized.
- XP is automatically awarded to the Helper.
- SMTP Email is sent to the Helper: "Your quest was confirmed! +(amount) XP."

Journey 4: Guildmaster Validation & Reward

1. Guildmaster (Admin) logs into the React Dashboard.
2. They see a global log of all quests to ensure no "illegal" or "scam" quests are posted.
3. Guildmaster has the power to "Flag" or "Delete" quests that violate community standards.
4. Dispute Resolution: If a Helper claims they finished but the Poster refuses to pay, the Guildmaster reviews the provided proof from either chat and can manually trigger the completion.

2.3 Feature List (MoSCoW)

MUST HAVE

1. Authentication & Security: User registration/login with JWT, BCrypt password hashing, and Google OAuth integration.
2. Quest Board (Core CRUD): Full Create, Read, Update, and Delete capabilities for Quests.
3. File Upload: Uploading pdf or image file to a quest.
4. Payment Gateway for paid quests.
5. Inspiration quotes API integration (Quotable) as flavor text.
6. Email (SMTP): Automated welcome and quest notification emails.
7. Role-Based Access Control: Distinct views and API restrictions for Adventurers (Users) and Guildmasters (Admins).
8. Mobile & Web Frontends: Responsive React Vite site and Kotlin Android app.

SHOULD HAVE

1. Quest-Linked Chat: Real-time messaging between Poster and Helper.

2. XP & Leveling System: Backend logic to calculate user rank based on completed quest difficulty.
3. Search & Filtering: Ability to filter quests by category (e.g., Tutoring, Manual Labor).
4. User Profile Dashboard: A view for users to see their current Level, XP progress, rank, and skills.

COULD HAVE

1. Quest Leaderboard: A global ranking of top adventurers in specific communities
2. Achievement Badges: Special icons for completing specific types of quests (e.g., "Scholar" for 5 tutoring quests).
3. Dark Mode: Themed UI toggle for both Web and Mobile.
4. Push Notifications: Sending alerts to the Android app when a nearby quest is posted.

WON'T HAVE

1. Real Money Transactions: Only Sandbox/Test mode will be implemented.
2. Advanced Analytics: Complex data visualization for "Grandmasters" is deferred.
3. Map API integration.
4. Multi-language Support: The initial system will be English-only.

2.4 Detailed Feature Specifications

Feature: Authentication & Identity

- **Screens:** Registration, Login, Profile View.
- **Security:** BCrypt hashing, JWT stateless tokens, and Google OAuth 2.0.
- **API Endpoints:**
    - POST /api/auth/register (Native signup)
    - POST /api/auth/login (Standard login)
    - POST /api/auth/google (Exchange Google token for System JWT)
    - GET /api/auth/me (Mandatory endpoint for current user context)

Feature: The Quest Board (Core CRUD)

- **Screens:** Quest Feed (Board), Post Quest Form, Quest Detail.
- **Integration:** Quotable API (External API). Fetches a random "Inspirational Quote" to display as "Heroes' Wisdom" on the board header.
- **Search/Filter:** Users can filter quests by Category (e.g., Physical, Digital, Academic) and Reward Range.
- **API Endpoints:**
    - GET /api/quests (Supports query params: ?category=...&keyword=...)
    - POST /api/quests (Creates quest; handles JSON data)
    - GET /api/quests/{id} (View specific quest details)
    - GET /api/wisdom (Backend proxy call to the External Quote API)

Feature: Quest Briefing (File Upload)

- **Screens:** Post Quest (Upload Section), Quest Detail (Attachment Link).
- **Functions:** Poster can attach a PDF, Doc, or Image file to provide detailed instructions or reference material for the quest.
- **Storage:** Files are stored in the server's local file system; unique file paths are linked to the Quest record in Supabase.
- **API Endpoints:**
    - POST /api/quests (Updated to Multipart/form-data to accept both text and file)
    - GET /api/quests/{id}/download (Stream the attached briefing file to the user)

Feature: Payments (Reward System)

- **Screens:** Checkout, Payment Success/Failure.
- **Integration:** Stripe Sandbox (Test Mode).
- **Persistence:** Every quest is linked to a transaction record in the payments table.
- **API Endpoints:**
    - POST /api/payments/create-session/{questId} (Initiates Stripe Checkout, if quest type is paid)
    - GET /api/payments/verify/{sessionId} (Updates payment status in database)

Feature: Quest Chat (Real-Time)

- **Screens:** Message Overlay (within Quest Detail)
- **Functions:** Private text communication between the Poster and the Adventurer who accepted the quest.
- **Implementation: WebSocket** (STOMP) for live updates
- **API Endpoints:**
    - WS /ws/chat (WebSocket connection point)
    - GET /api/quests/{id}/messages (Fetch message history)

Feature: Guildmaster Global Control (Admin)

- **Access Control:** Global RBAC. Requires ROLE_GUILDMASTER.
- **Functions:**
    - Guild Oversight: Create or delete Guilds.
    - Content Moderation: Remove any quest or message across the entire system.
- **API Endpoints:**
    - POST /api/admin/guilds (Create)

- DELETE /api/admin/quests/{id} (Moderation)

2.5 Acceptance Criteria

AC-1: User Onboarding & Auto-Enrollment

Given I am a new user registering for the first time,

When my account is successfully created in the database,

Then the system must automatically assign me an ACTIVE membership to the "Global Square" guild,

And I must be able to view the Global Quest Board immediately

AC-2: Integrated User Onboarding (Security & Email)

Given I am a new user on the Web or Android app,

When I register via a native form or Google OAuth,

Then the system must hash my password using BCrypt (if native) and save my record in the users table,

And the system must return a valid JWT to my device,

And I must receive a "Welcome to the Guild" SMTP Email notification.

AC-3: Quest Commissioning (Paid)

Given I am a logged-in Adventurer (User) within a specific Guild,

When I create a quest and optionally upload a Briefing File (PDF/Image),

And I set the quest type to “paid”,

And I complete the Stripe Sandbox payment for the "reward,"

Then the quest must appear on that Guild’s board with the attachment accessible for download,

And a record must be created in the payments table linked to the quest.

AC-4: Quest Commissioning (Volunteer)

Given I am a logged-in Adventurer (User) within a specific Guild,

When I create a quest and optionally upload a Briefing File (PDF/Image),

And I set the quest type to “volunteer”,

Then the quest must appear on that Guild’s board with the attachment accessible for download,

AC-5: Community Interaction (Chat & Wisdom)

Given I am an Adventurer browsing a Guild quest board,

When I view the board, the system must fetch and display "Heroes' Wisdom" from the Quotable API,

And when I accept a quest, I must be able to send real-time messages to the Poster via WebSockets,

Then all messages must be persisted in the messages table for future reference.

AC-6: Validation & Rewards (Logic & RBAC)

Given I am the poster of a quest or a Guildmaster (Admin),

When I review the completed work and click "Approve,"

Then the system must update the Helper’s XP and Level in the users table,

And an SMTP Email must be sent to the Helper confirming their reward,

And the quest must transition to the COMPLETED state in the database.

AC-7: Global Moderation (RBAC)

Given I am logged in as the Guildmaster,

When I navigate to any Guild and identify a Quest that violates the terms of service (e.g., "Illegal Activity"),

Then I must be able to delete that Quest immediately,

And the system must void the associated "Reward" record and notify the Poster via SMTP Email.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API Response Time: ≤ 2 seconds for 95% of standard REST requests.
- External API Latency: The "Heroes' Wisdom" (Quotable API) must fetch and render within 1.5 seconds of page load.
- File Upload/Download Latency: Quest briefings (PDF/Images up to 10MB) must finish uploading within 8 seconds on standard 4G connections.
- Concurrent Users: Support 100 concurrent adventurers interacting across different Guilds.
- Real-time Latency: Chat messages must be delivered within 1 second using WebSockets (STOMP).

3.2 Security Requirements

- Encryption: HTTPS/TLS 1.3 for all data in transit.
- Identity: JWT-based stateless authentication with a 24-hour expiration.
- Social Auth: Secure handling of Google OAuth 2.0 tokens; internal IDs must be linked to Google sub (subject) IDs.
- Data Protection: Password hashing using BCrypt with 12 salt rounds.
- Scoped RBAC: Server-side verification of roles (ROLE_ADVENTURER, ROLE_GUILDMASTER) before accessing restricted endpoints.

3.3 Compatibility Requirements

- Web Frontend (React Vite): Chrome, Firefox, Safari, and Edge. Optimized for Guildmasters to manage guilds and oversee the system.
- Mobile Frontend (Kotlin): Android API Level 26+ (Android 8.0 Oreo). Optimized for Adventurers to browse and accept quests on the go.
- Responsive Design: Fully responsive from 320px (mobile) to 2560px (desktop).

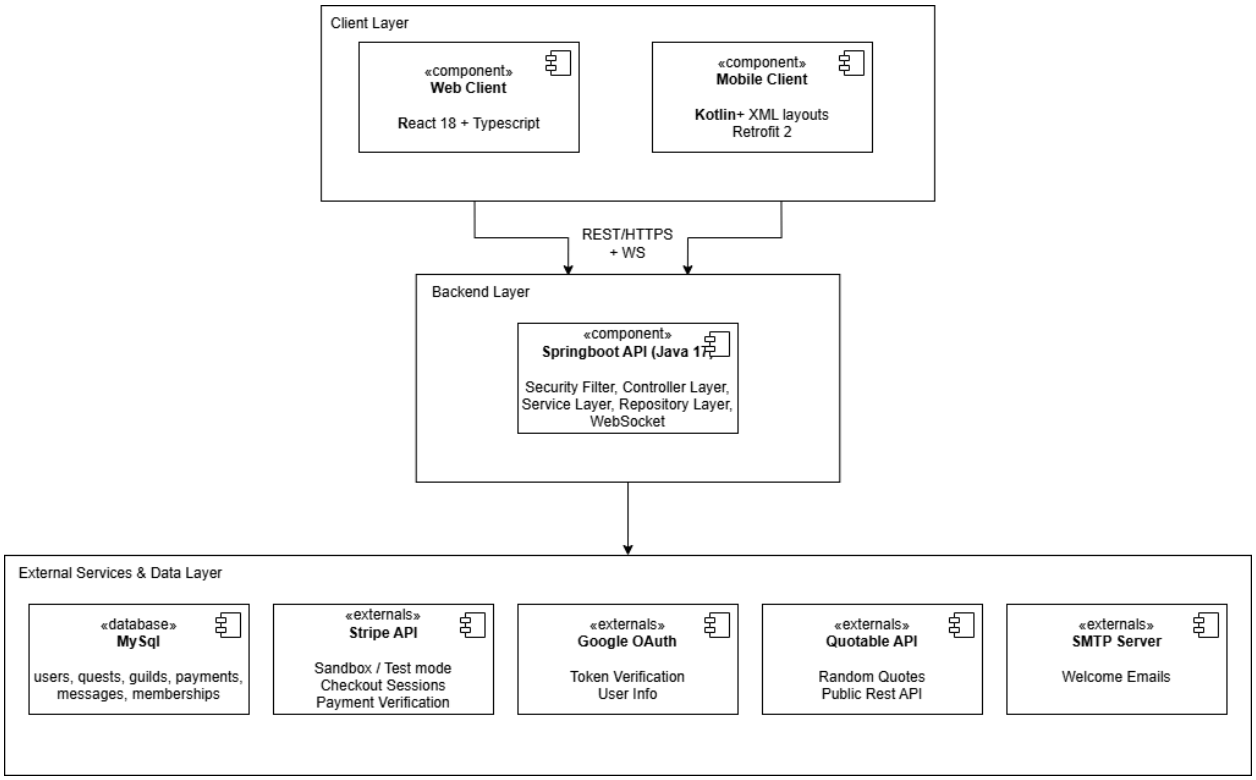
3.4 Usability Requirements

- Onboarding Speed: New users should be able to "Continue with Google" and join their first Guild within 2 minutes.
- Accessibility: Following WCAG 2.1 guidelines.

- Thematic Consistency: Error messages and success alerts should use "Guild" terminology (e.g., "Quest Commission Failed" instead of "Form Submission Error").
- Mobile-First Design: Minimum 48x48px touch targets for core actions like "Accept Quest" or "Open Briefing."

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram



Technology Stack:

- **Backend:** Java 19, Spring Boot 3.5.11, Spring Security (JWT), Spring Data JPA

- **Database:** Supabase (PostgreSQL)
- **Web Frontend:** React 18 (Vite), TypeScript, Tailwind CSS, Axios
- **Mobile:** Kotlin, XML based layouts, Retrofit 2
- **Build Tools:** Maven (Backend), npm (Web), Gradle (Android)
- **Deployment:** Render (Web), APK (Mobile)

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

- **Base URL:** https://api.guildhall.app/api/v1
- **Format:** JSON for all requests/responses
- **Authentication:** Bearer token (JWT) in Authorization header
- **File Upload endpoints use:** multipart/form-data
- **WebSocket Connection:** wss://api.guildhall.app/ws
- **Response Structure:**

```
json
{
  "success": boolean,
  "data":    object | null,
  "error": {
    "code":    string,
    "message": string,
    "details": object | null
  },
  "timestamp": string   // ISO-8601, e.g. "2026-03-01T10:30:00Z"
}
```

5.2 Endpoint Specifications

Authentication Endpoints

POST /auth/register

```
Body:  {email, password, confirmPassword, username}

Response: {user: {id, email, username, level, xp}, token}
```

POST /auth/login

```
Body:  {email, password}

Response: {user: {id, email, username, role, level, xp}, token}
```

POST /auth/google *(Google OAuth — exchange ID token for GuildHall JWT)*

```
Body:  {googleIdToken: string}

Response: {user: {id, email, username, role}, token, isNewUser: boolean}
```

POST /auth/logout

```
Headers:  Authorization: Bearer {token}

Response: {message: "Logged out successfully"}
```

GET /auth/me *(Authenticated)*

```
Headers:  Authorization: Bearer {token}

Response: {user: {id, email, username, role, level, xp, rank}}
```

Guild Endpoints

GET /guilds

```
Query: ?page=1&limit=20&search=keyword

Response: {guilds: [{id, name, description, memberCount, createdAt}], pagination: {...}}
```

GET /guilds/{id}

```
Response: {guild: {id, name, description, memberCount, questCount, createdAt}}
```

POST /guilds *(Guildmaster only)*

```
Headers:  Authorization: Bearer {token}
Body:   {name, description}

Response: {guild: {id, name, description, createdAt}}
```

DELETE /guilds/{id} *(Guildmaster only)*

```
Headers:  Authorization: Bearer {token}

Response: {message: "Guild  disbanded"}
```

POST /guilds/{id}/join *(Authenticated)*

```
Headers:  Authorization: Bearer {token}

Response: {message: "Joined guild", membership: {guildId, userId, joinedAt}}
```

DELETE /guilds/{id}/leave *(Authenticated)*

```
Headers:  Authorization: Bearer {token}

Response: {message: "Left guild"}
```

GET /guilds/{id}/members *(Authenticated)*

```
Headers:  Authorization: Bearer {token}

Response: {members: [{id, username, level, rank, joinedAt}], pagination: {...}}
```

Quest Endpoints

GET /guilds/{guildId}/quests *(Authenticated — member only)*

```
Query:   ?page=1&limit=20&category=Academic&status=OPEN&keyword=...

Response: {quests: [{id, title, category, reward, status, poster, createdAt}], pagination: {...}}
```

POST /guilds/{guildId}/quests *(Authenticated — multipart/form-data)*

```
Headers:  Authorization: Bearer {token}  |  Content-Type: multipart/form-data
Body:  {title, description, category, reward} + file? (PDF/Image, optional)

Response: {quest: {id, title, category, quest_type, reward, status: PENDING_PAYMENT, attachmentUrl?}}
```

GET /guilds/{guildId}/quests/{id} *(Authenticated — member only)*

```
Response: {quest: {id, title, description, category, reward, status, poster, helper?, attachmentUrl?,
createdAt}}
```

PUT /guilds/{guildId}/quests/{id} *(Quest poster only — only when status = OPEN)*

```
Headers:  Authorization: Bearer {token}
Body:  {title?, description?, category?}

Response: {quest: {...}}
```

DELETE **/guilds/{guildId}/quests/{id}** *(Quest poster or Guildmaster)*

```
Headers:  Authorization: Bearer {token}

Response: {message: "Quest removed"}
```

GET **/guilds/{guildId}/quests/{id}/download** *(Authenticated — streams briefing file)*

```
Response: File stream (PDF/Image)
```

POST **/guilds/{guildId}/quests/{id}/accept** *(Authenticated — status changes OPEN → PENDING)*

```
Headers:  Authorization: Bearer {token}

Response: {quest: {id, status: PENDING, helper: {id, username}}}
```

POST **/guilds/{guildId}/quests/{id}/complete** *(Quest poster — status changes PENDING → COMPLETED, triggers XP award & email)*

```
Headers:  Authorization: Bearer {token}

Response: {quest: {id, status: COMPLETED}, xpAwarded: number}
```

Heroes' Wisdom Endpoint (External API Integration)

GET **/wisdom** *(Public — backend proxy to Quotable API)*

```
Response: {quote: {content: string, author: string}}
```

Payment Endpoints (Stripe Sandbox)

POST **/payments/create-session/{questId}** *(Authenticated — initiates Stripe Checkout for reward)*

```
Headers:  Authorization: Bearer {token}

Response: {sessionId: string, checkoutUrl: string}
```

GET **/payments/verify/{sessionId}** *(Called after Stripe redirect — updates DB, sets quest OPEN)*

```
Headers:  Authorization: Bearer {token}

Response: {payment: {id, questId, amount, status, createdAt}}
```

GET **/payments/history** *(Authenticated — own payment records)*

```
Headers:  Authorization: Bearer {token}

Response: {payments: [{id, questId, amount, status, createdAt}]}
```

Quest Chat Endpoints (Real-Time)

WS **/ws/chat** *(WebSocket — STOMP protocol, requires JWT in handshake header)*

```
Response: Bidirectional message stream
```

GET **/guilds/{guildId}/quests/{id}/messages** *(Authenticated — Poster or Helper of that quest only)*

```
Headers:  Authorization: Bearer {token}

Response: {messages: [{id, senderId, content, sentAt}]}
```

User Profile Endpoints

GET /users/{id}/profile *(Authenticated)*

```
Response: {user: {id, username, level, xp, rank, completedQuests, joinedCommunities}}
```

GET /users/{id}/quest-history *(Authenticated)*

```
Query: ?page=1&limit=20&role=helper

Response: {quests: [{id, title, status, role: poster|helper, completedAt?}], pagination: {...}}
```

Guildmaster / Admin Endpoints

GET /admin/quests *(Guildmaster only — global quest log across all guilds)*

```
Query:   ?page=1&limit=20&guildId=...&status=OPEN
Headers:  Authorization: Bearer {token}

Response: {quests: [{id, title, guildId, status, poster, createdAt}], pagination: {...}}
```

DELETE /admin/quests/{id} *(Guildmaster only — moderation delete; triggers reward refund notification)*

```
Headers:  Authorization: Bearer {token}

Response: {message: "Quest removed by Guildmaster", posterNotified: true}
```

GET /admin/payments *(Guildmaster only — all Stripe transactions)*

```
Headers:  Authorization: Bearer {token}

Response: {payments: [{id, questId, amount, status, createdAt}], pagination: {...}}
```

POST **/admin/users/{id}/ban** *(Guildmaster only — restrict user from system)*

```
Headers:  Authorization: Bearer {token}
Body:   {reason}

Response: {message: "User restricted"}
```

POST **/admin/quests/{id}/resolve** *(Guildmaster only — manually complete a disputed quest)*

```
Headers:  Authorization: Bearer {token}

Response: {quest: {id, status: COMPLETED}, xpAwarded: number}
```

5.3 Error Handling

HTTP Status Codes

- 200 OK — Successful request
- 201 Created — Resource created
- 400 Bad Request — Invalid input or validation failure
- 401 Unauthorized — Authentication required or token invalid
- 403 Forbidden — Insufficient role permissions
- 404 Not Found — Resource does not exist
- 409 Conflict — Duplicate resource (e.g. username already taken)
- 422 Unprocessable Entity — Quest state transition not allowed (e.g. accepting a PENDING quest)
- 500 Internal Server Error — Unexpected server error

Error Code Examples

```
// AUTH-001: Invalid credentials
{
  "success": false,
  "data": null,
  "error": {
        "code":   "AUTH-001",
        "message": "Invalid credentials",
        "details": "Email or password is incorrect"
  },
  "timestamp": "2026-03-01T10:30:00Z"
```

```
}
```

```
// VALID-001: Validation failure
{
  "success": false,
  "data": null,
  "error": {
          "code":   "VALID-001",
          "message": "Validation failed",
          "details": {
          "title":  "Quest title is required",
          “quest_type” : “Quest type is required”,
          "reward": "(optional) Reward must be a positive number"
          }
  },
  "timestamp": "2026-03-01T10:30:00Z"

}
```

```
// QUEST-002: Invalid state transition
{
  "success": false,
  "data": null,
  "error": {
          "code":   "QUEST-002",
          "message": "Quest is not available",
          "details": "Only OPEN quests can be accepted"
  },
  "timestamp": "2026-03-01T10:30:00Z"

}
```

Common Error Codes

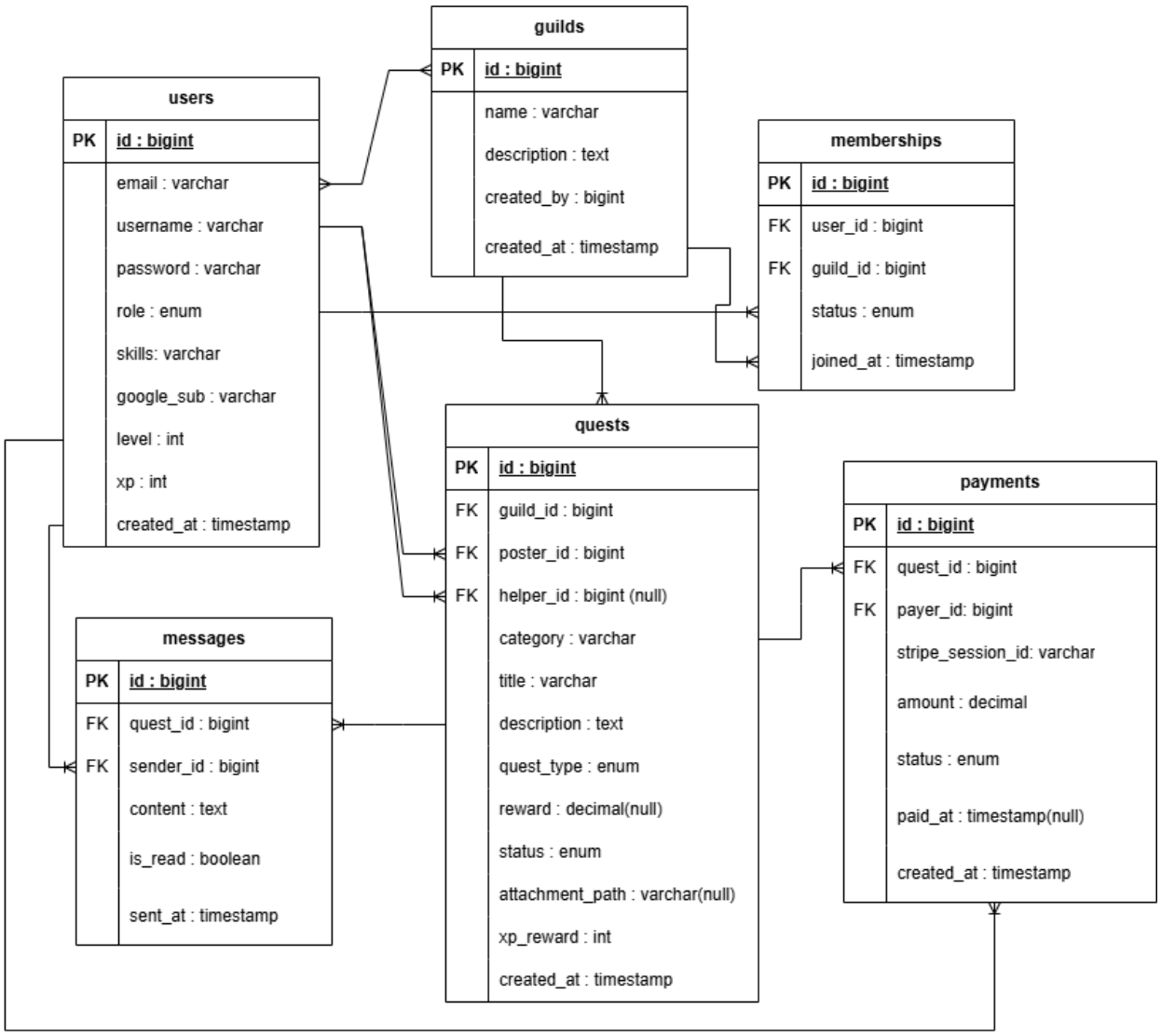
Error Code	Description

AUTH-001	Invalid email or password
AUTH-002	JWT token has expired — re-login required
AUTH-003	Insufficient permissions for this action
AUTH-004	Google OAuth token invalid or revoked
VALID-001	Request body failed validation — see details field
QUEST-001	Quest not found
QUEST-002	Invalid quest state transition (e.g. accepting a non-OPEN quest)
QUEST-003	Only the quest Poster can perform this action
GUILD-001	Guild not found
GUILD-002	User is not a member of this guild
PAYMENT-001	Stripe session creation failed
PAYMENT-002	Payment not verified — quest remains unpublished
FILE-001	File type not allowed (only PDF/JPG/PNG)
FILE-002	File exceeds 10MB size limit

DB-001	Resource not found
DB-002	Duplicate entry — resource already exists
SYSTEM-001	Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram



Detailed Relationships:

- **Many-to-Many:** users ↔ guilds (resolved via the memberships bridge table)
- **One-to-Many:** guilds → quests (each guild hosts many quests)
- **One-to-Many:** users → quests as poster_id (a user can post many quests)
- **One-to-Many:** users → quests as helper_id (a user can help on many quests)
- **One-to-Many:** quests → payments (a quest has one payment record; modeled one-to-many for refund/retry scenarios)
- **One-to-Many:** users → payments as payer_id (a user can have multiple payment records)
- **One-to-Many:** quests → messages (a quest chat thread contains many messages)
- **One-to-Many:** users → messages as sender_id (a user can send many messages)
- **One-to-Many:** guilds→ memberships (each guild has many membership records)
- **One-to-Many:** users → memberships (each user has many membership records across different guilds)

Key Tables:

1. **users —** Adventurer and Guildmaster accounts, XP, level, and authentication data
2. **guilds—** User-created groups that contain and scope all quests
3. **memberships —** Bridge table resolving the many-to-many relationship between users and guilds
4. **quests —** The core business entity; scoped to a guild with poster, helper, status, and bounty
5. **payments —** Stripe payment records linked to each quest's bounty
6. **messages —** Persistent chat messages between Poster and Helper within a quest

Table Structure Summary:

- **users:** id, email, username, password, role, skills, google_sub, level, xp, created_at
- **guilds:** id, name, description, created_by, created_at
- **membership:** id, user_id, guild_id, status, joined_at

- **quests:** id, guild_id, poster_id, helper_id, category, title, description, reward, status, attachment_path, xp_reward, created_at
- **payments:** id, quest_id, payer_id, stripe_session_id, amount, status, paid_at, created_at
- **messages:** id, quest_id, sender_id, content, is_read, sent_at

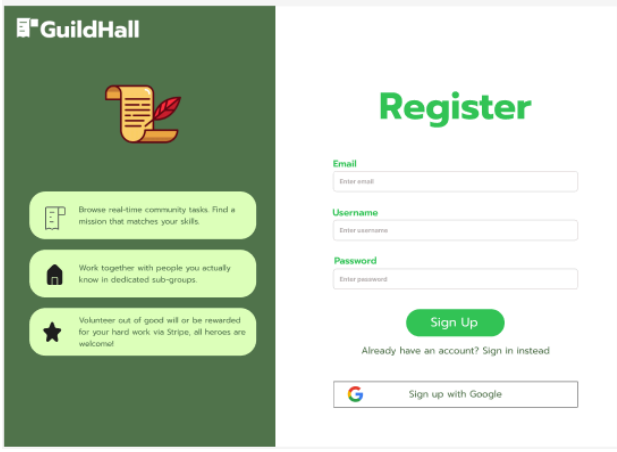
7.0 UI/UX DESIGN

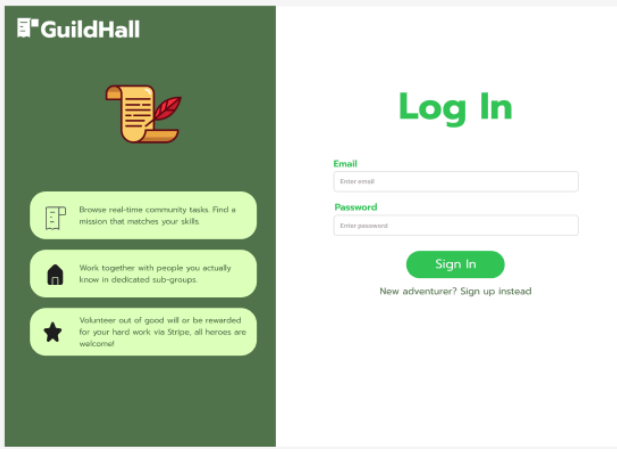
7.1 Web Application Wireframes

https://www.figma.com/design/6ax9AqEpcKSSoRqOkdJ1eq/GuildHall?node-id=0-1&t=9yxjfdkrZDXB3RWa-1

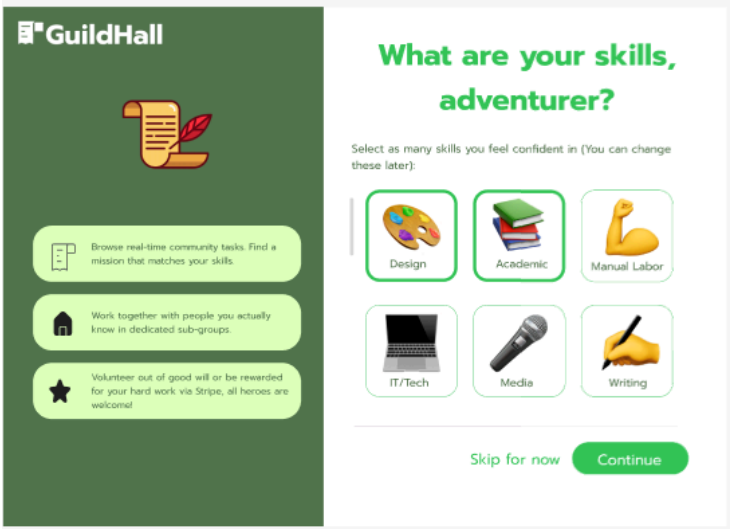
Landing Homepage

Sign Up and Sign In

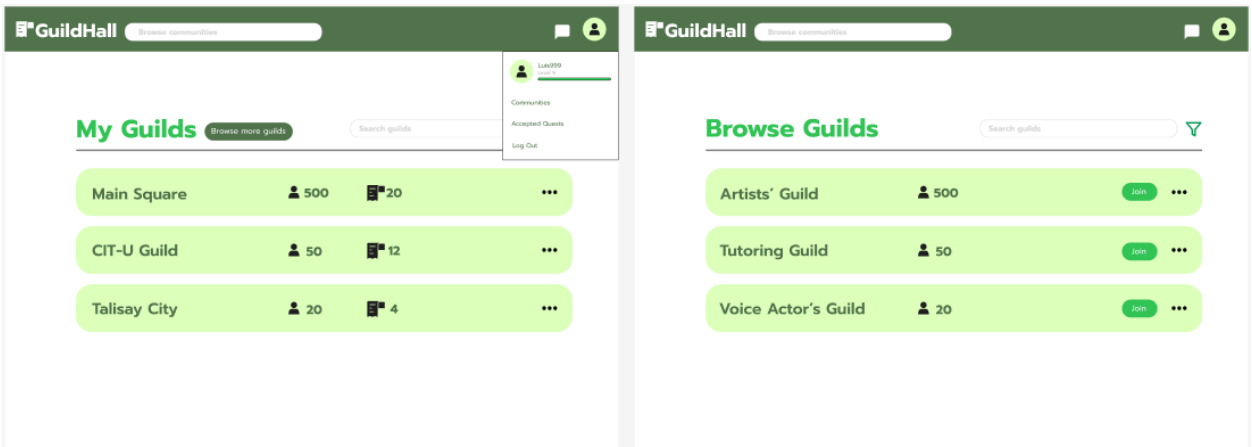




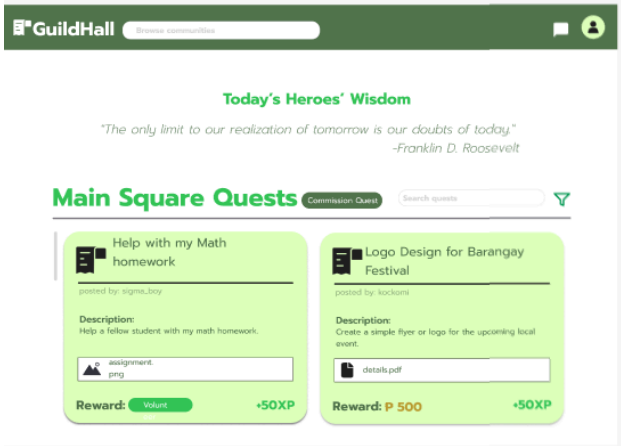
First time skills selection



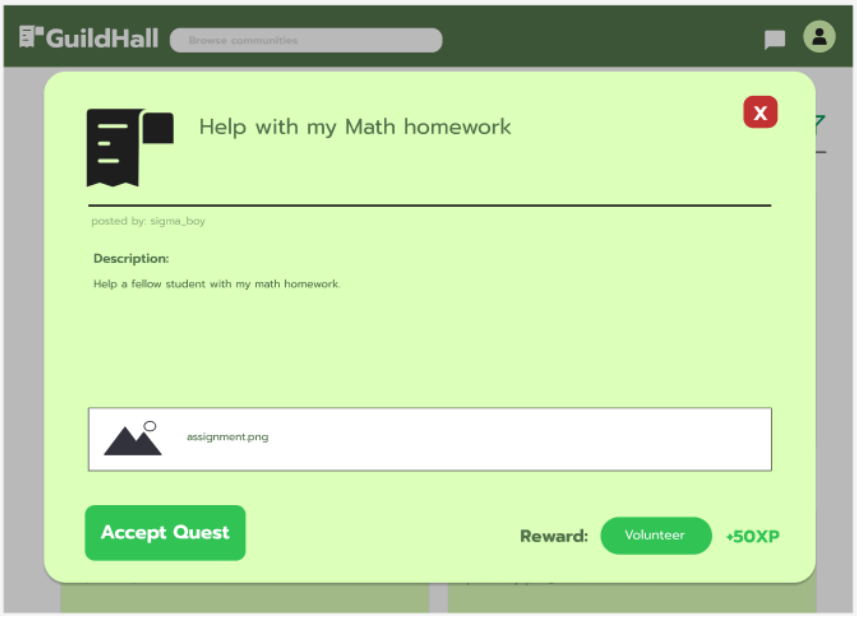
My Communities and Browse Communities



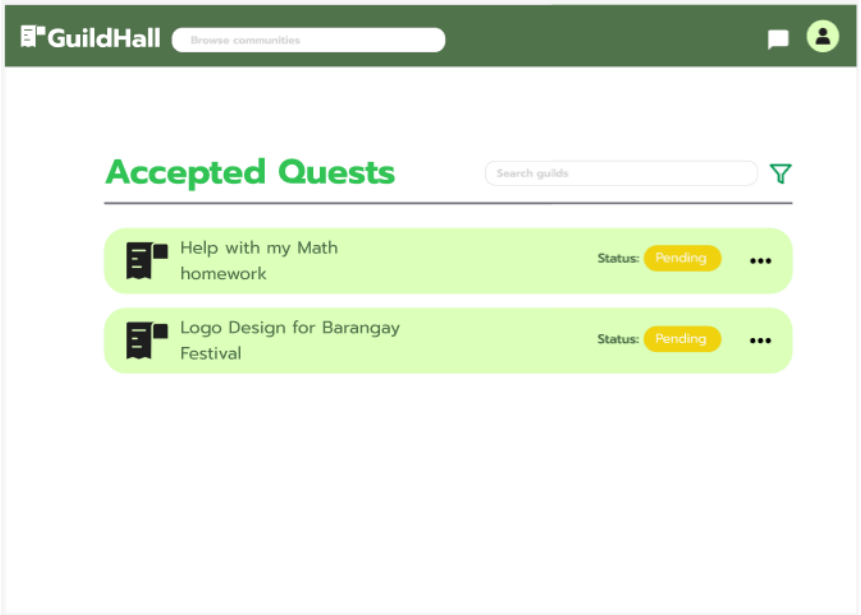
Quest Board



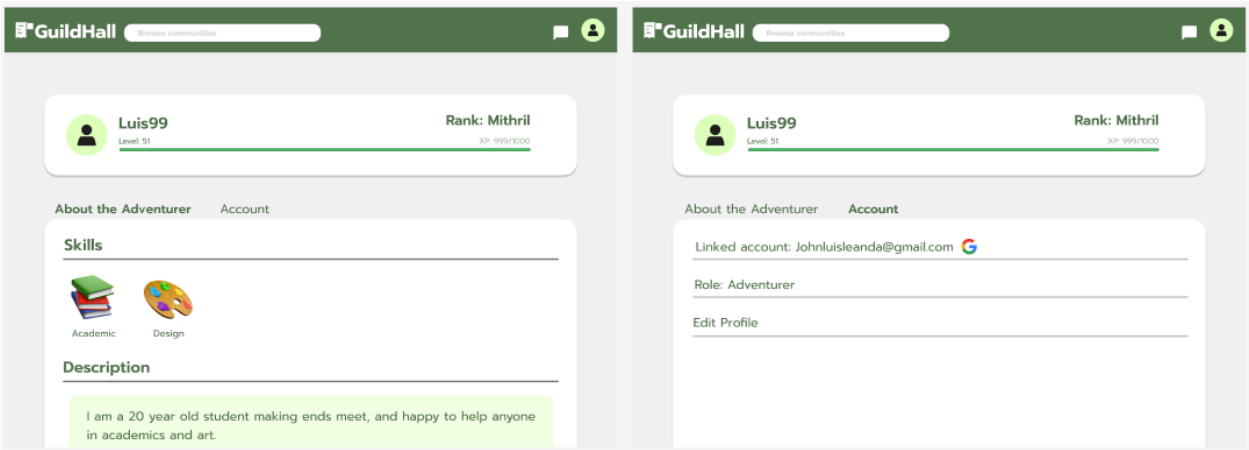
Quest Details



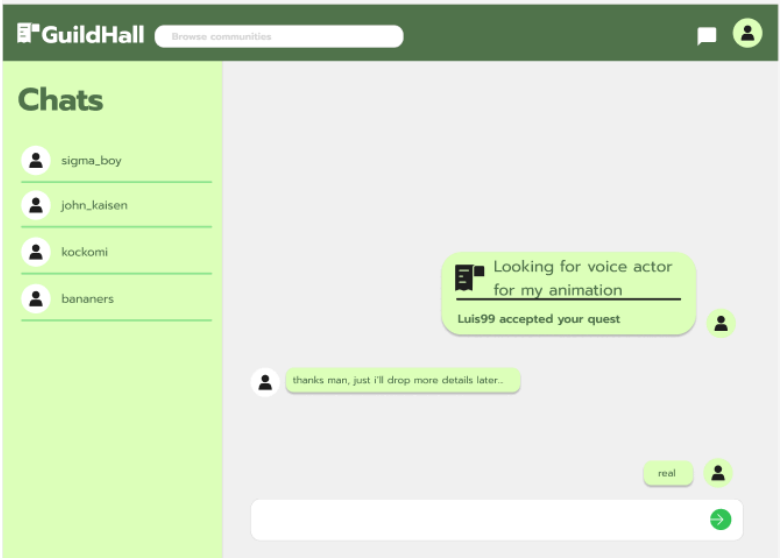
Accepted Quests



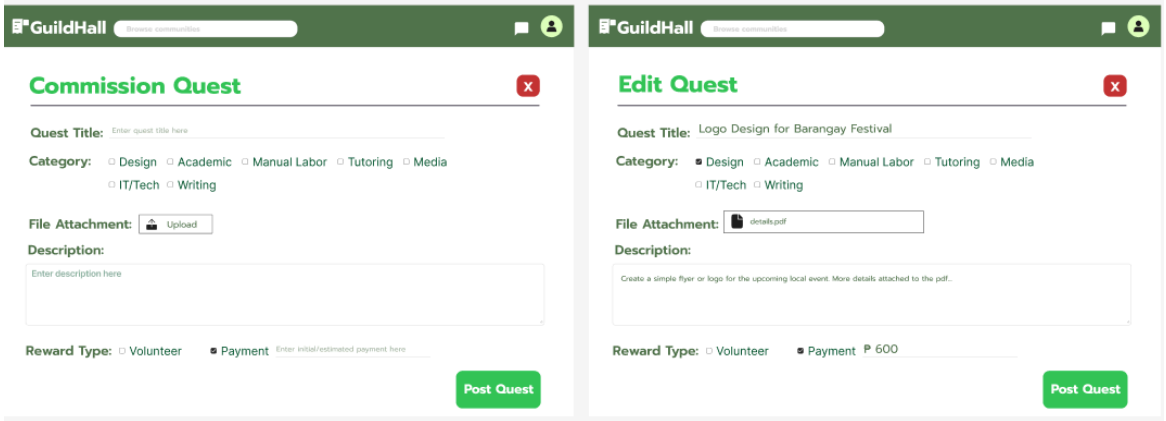
Profile



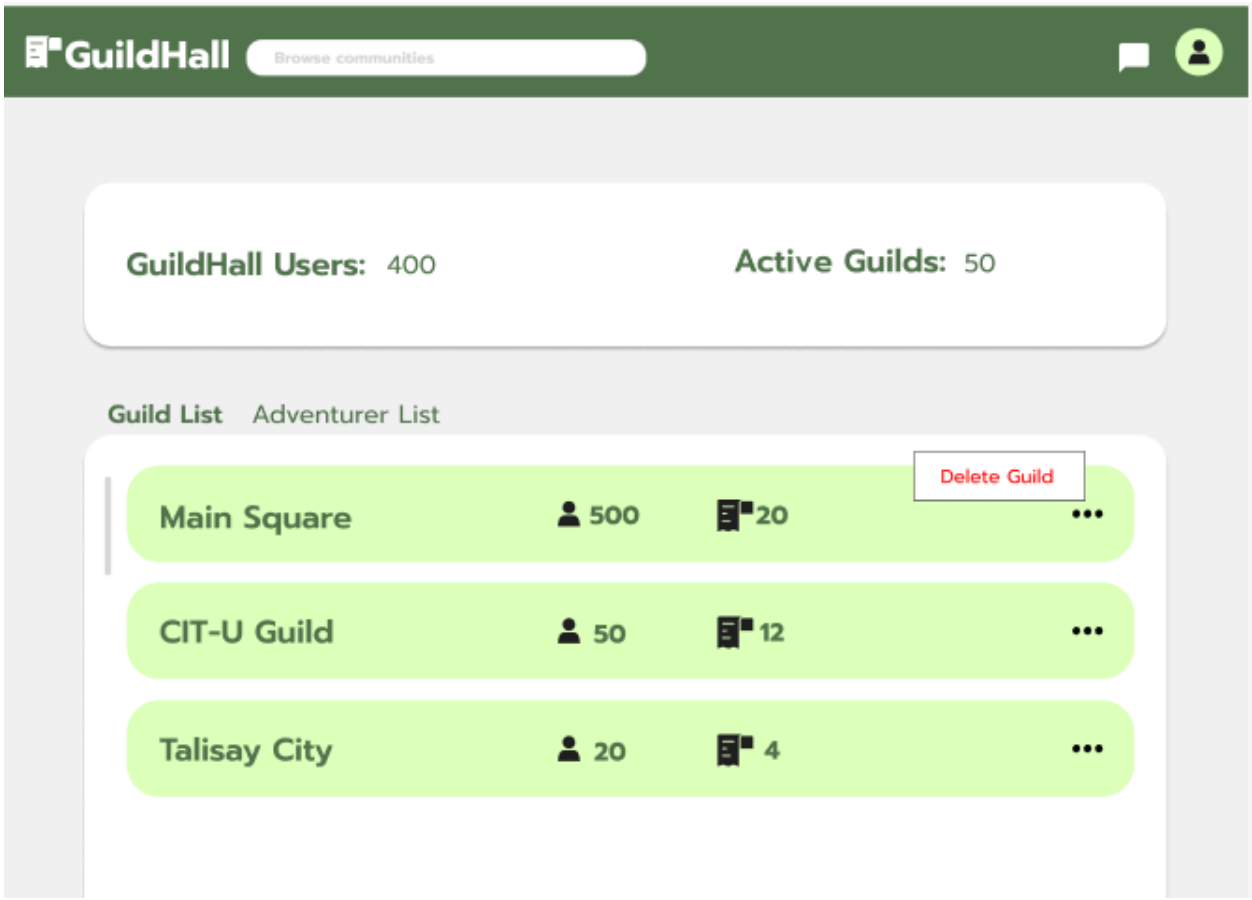
Chat



Commission Quest & Edit Quest

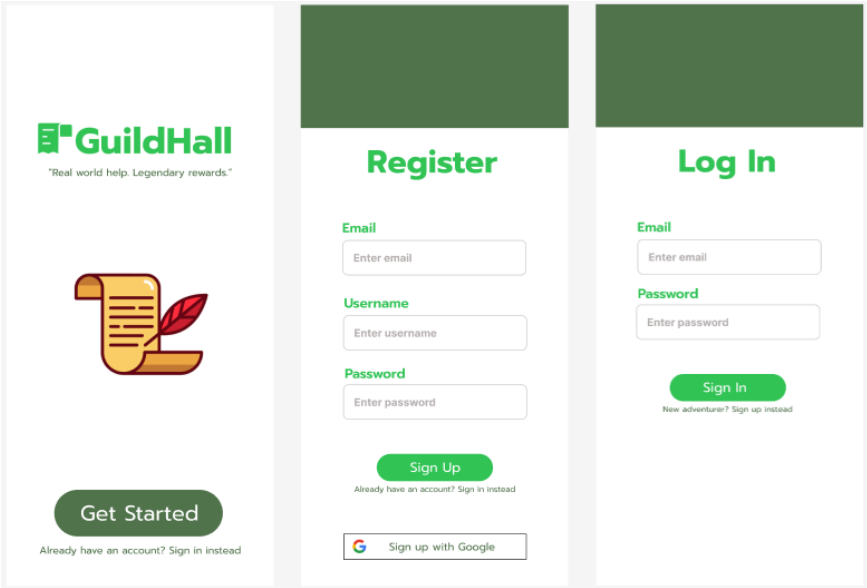


Admin Dashboard

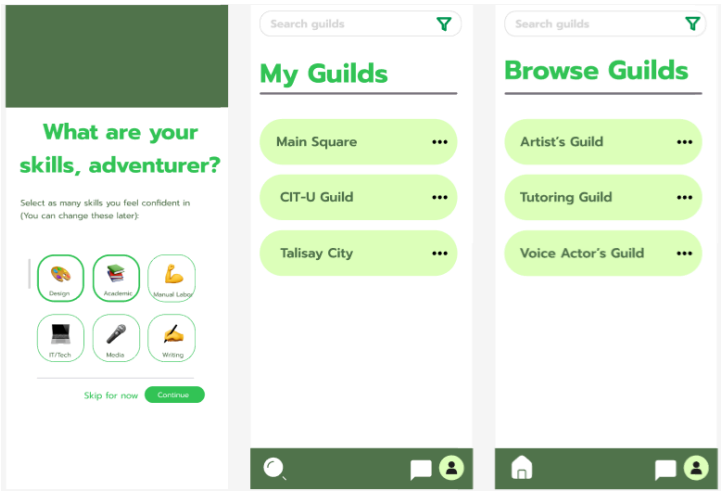


7.2 Mobile Application Wireframes

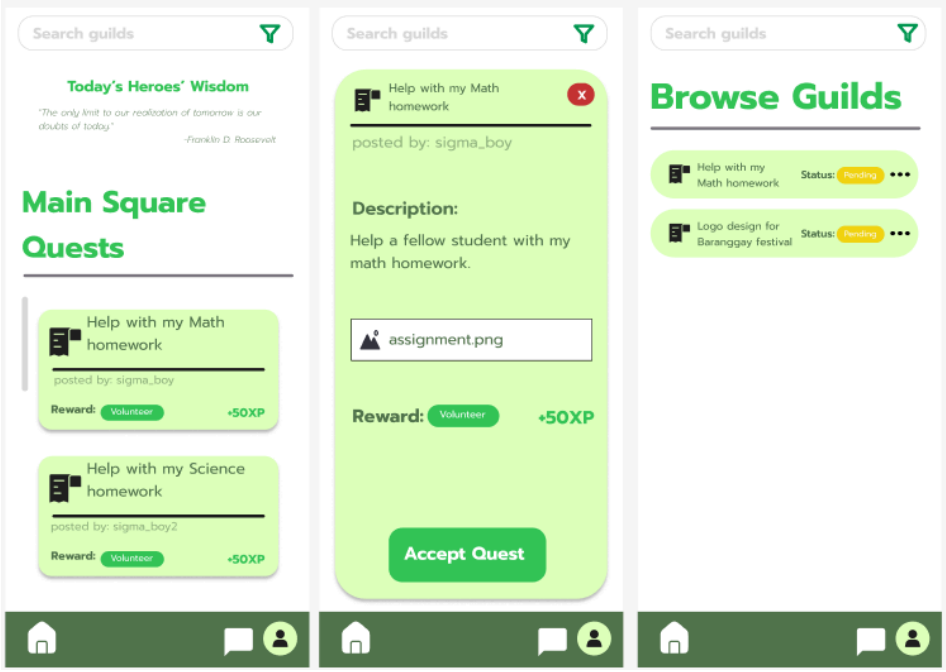
Landing, Register, Login



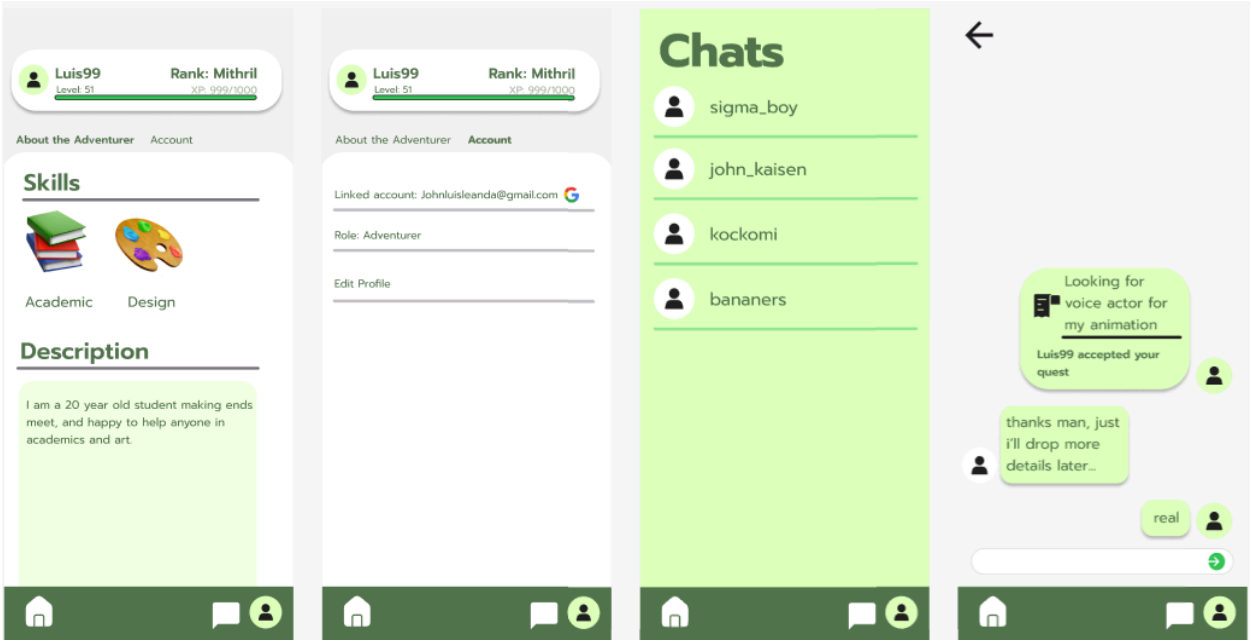
First time Skills selection, Community Page, Browse Communities



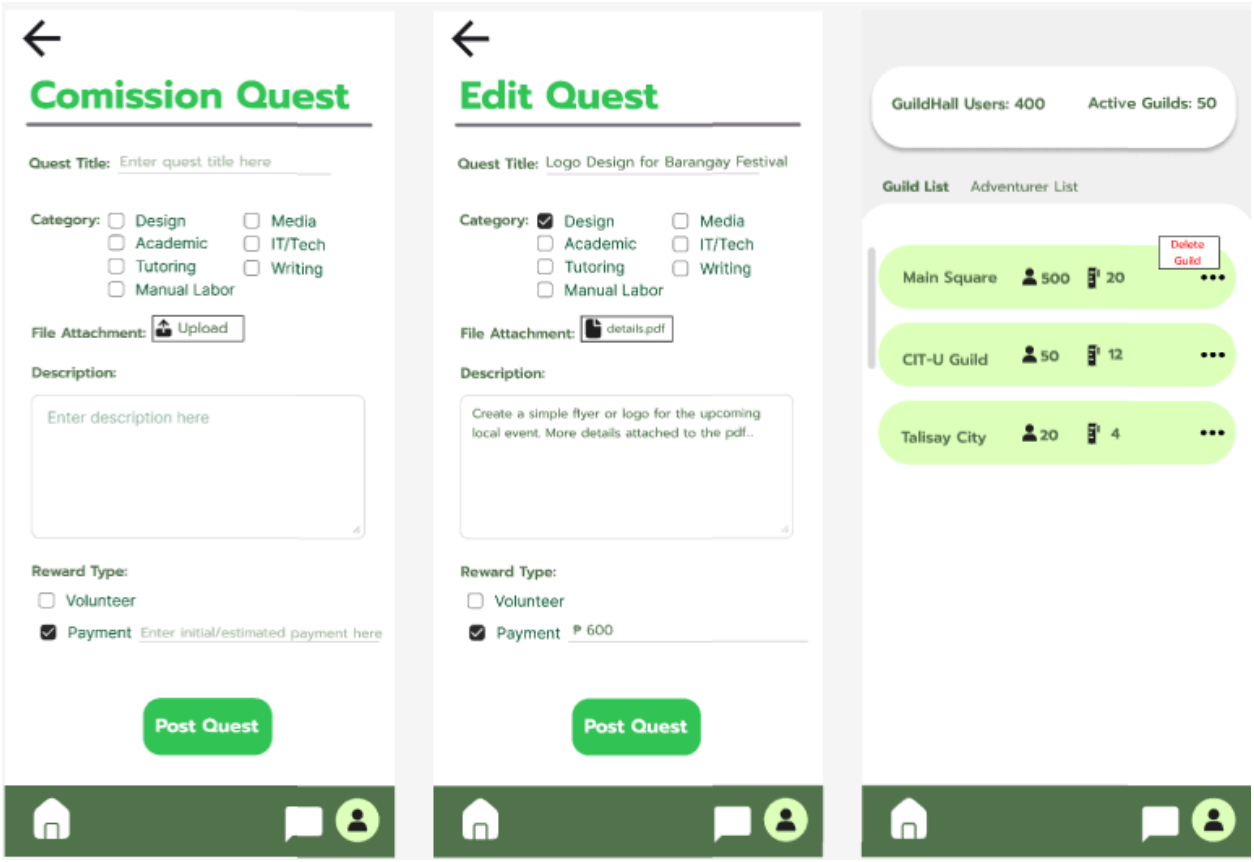
Quest Board, Quest Details, Accepted Quests



Profile, Chat



Commission Quest, Edit Quest, Admin Dashboard



Mobile-Specific Features:

- Touch-optimized buttons (min 48x48px)
- Gesture support (swipe, pull-to-refresh)
- Simplified forms for mobile input

Design System:

- **Colors:** Primary (#34C759), Secondary (#DDFFBC), Tertiary (52734D), Error and Cancel (#C73434), Negative Space (#FFFFFF)
- **Typography:** Font: Prompt
- **Spacing:** 8px grid system
- **Components:** Consistent buttons, inputs, cards, modals
- **Responsive:** Mobile-first approach, breakpoints at 640px, 768px, 1024p

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

Week 1: Requirements & Architecture

Day 1-2: Project setup and theme finalization (Adventurer/Guild terminology)

Day 3-4: Finalize FRS (Quest Board, Chat, Stripe) and NFRs

Day 5-7: System architecture design (Web, Mobile, Backend, Postgre)

Week 2: Detailed Design

Day 1-2: API specification (REST endpoints for Quests and Guilds)

Day 3-4: Database design (ERD with 6 tables: Users, Guilds, Memberships, Quests, Payments, Messages)

Day 5-6: UI/UX wireframes in Figma (Quest Board, Adventurer Profile, Chat)

Day 7: Implementation plan finalization

Phase 2: Backend Development (Week 3-4)

Week 3: Foundation

Day 1: Spring Boot setup with Security, JPA, and WebSocket dependencies

Day 2: Database configuration and Entity classes

Day 3: JWT Authentication & Google OAuth 2.0 implementation

Day 4: User management & XP/Leveling logic

Day 5: Guild & Membership management

Week 4: Core Features

Day 1: Quest CRUD operations (Create, Browse, Accept)

Day 2: Stripe Payment integration (Bounty Escrow)

Day 3: WebSocket Chat & Automated System Messages

Day 4: Quotable API integration (Heroes' Wisdom) & Error Handling

Day 5: API documentation and testing

Phase 3: Web Application (Week 5-6)

Week 5: Frontend Foundation

Day 1: React setup with TypeScript and Tailwind CSS

Day 2: Authentication pages (Google Login, Register)

Day 3: Main Quest Board (Filtering by Guild/Category)

Day 4: Quest Detail Page & Briefing Download

Day 5: Adventurer Profile (XP Bar, Skill Badges)

Week 6: Complete Web Features

Day 1: Real-time Chat interface (WebSocket client)

Day 2: Quest Posting & Stripe Checkout flow

Day 3: Guildmaster Admin Dashboard

Day 4: Responsive design polish

Day 5: API integration and end-to-end testing

Phase 4: Mobile Application (Week 7-8)

Week 7: Android Foundation

Day 1: Android Studio setup & XML Layout structures

Day 2: Authentication Screens

Day 3: Quest Browsing

Day 4: API Service Layer

Day 5: Briefing Download & File Handling

Week 8: Complete Mobile App

Day 1: Active Quest Management & Accept logic

Day 2: Mobile Chat implementation

Day 3: UI polish

Day 4: Testing on physical Android device/emulator

Day 5: APK generation and documentation

Phase 5: Integration & Deployment (Week 9-10)

Week 9: Integration Testing

Day 1: Cross-platform testing

Day 2: Bug fixes (specifically Chat and Stripe webhooks)

Day 3: Security review

Day 4: Performance testing

Day 5: Documentation finalization

Week 10: Deployment

Day 1: Web app deployment (Render)

Day 2: Mobile APK distribution (GitHub Release)

Day 3: Final system health check

Day 4: Project submission & Presentation prep

Milestones:

- **M1 (End Week 2):** All design documents complete
- **M2 (End Week 4):** Backend API fully functional
- **M3 (End Week 6):** Web application complete
- **M4 (End Week 8):** Mobile application complete
- **M5 (End Week 10):** Full system deployed and integrated

Critical Path:

1. Authentication system (Week 3)
2. Quest Status State Machine (Week 4)
3. Real-time WebSocket Sync (Week 6-8)
4. Cross-platform testing (Week 9)

Risk Mitigation:

- Start with simplest working version of each feature
- Test integration points early and often
- Keep backup of working versions
- Focus on core functionality before enhancements
- Test Stripe and WebSockets in isolation before adding them to the main UI
- Focus on the "Global Square" first before adding private Guilds